MANUAL PRÁCTICO | INFRAESTRUCTURA COMO CÓDIGO

Terraform + AWS

Guía de laboratorio paso a paso

Configuración de AWS CLI, usuario IAM dedicado, proyecto Terraform y creación de un bucket S3. Incluye errores reales encontrados y su solución.

Laboratorio realizado el 18 de octubre de 2026

Región us-east-1 | AWS CLI 2.36.24 | Provider hashicorp/aws 6.69.1

Contenido

Datos del autor 3

Resumen ejecutivo 3

1. Competencias aplicadas en el laboratorio 4

2. Levantamiento de información del laboratorio 5

3. Ejecución del despliegue 6

3.1 Usuario IAM dedicado 6

3.2 Configuración de AWS CLI 6

3.3 Proyecto Terraform 6

3.4 Flujo de trabajo de Terraform 7

3.5 Verificación 7

4. Registro de incidencias 9

5. Buenas prácticas 10

6. Cierre y siguientes pasos 10

Referencia rápida 11

Datos del autor

Desarrollado por	Nick Uchasara
Formación	Estudiante de Redes y Comunicaciones, 10.º ciclo
Área de interés	Infraestructura y servicios cloud
Laboratorio realizado	18 de octubre de 2026 | Región us-east-1 | AWS CLI 2.36.24 | Provider hashicorp/aws 6.69.1

Resumen ejecutivo

Realicé de forma autodidacta un laboratorio en el que configuré AWS CLI con un usuario IAM dedicado (sin usar el usuario raíz), inicialicé un proyecto de Terraform y creé un bucket de Amazon S3 en la región us-east-1. Verifiqué el resultado en la consola de AWS y por línea de comandos, y documenté cada paso, los dos errores que aparecieron con su causa y solución, y las buenas prácticas de seguridad aplicables.

Elemento	Valor
Recurso creado	aws_s3_bucket.example
Nombre del bucket	my-tf-test-nicktf18102026
Región	us-east-1 (Norte de Virginia)
Etiquetas	Environment = Dev, Name = My bucket
Usuario IAM	terraform-lab (grupo terraform-lab-admins)
Resultado final	Apply complete! 1 added

1. Competencias aplicadas en el laboratorio

Resumen de las competencias técnicas y de organización que se pusieron en práctica durante el laboratorio, con la sección donde se documentan.

Competencia	Cómo se aplicó	Sección
Documentación técnica	Registro paso a paso con capturas, tabla de errores, buenas prácticas y referencia rápida.	Todo el documento
Levantamiento de información	Definición previa de objetivos, requisitos, costos y resultado esperado.	2
Seguimiento de despliegues	Flujo ordenado init, fmt, validate, plan, apply, con verificación en consola y por CLI.	3
Gestión de incidencias	Errores registrados con síntoma, causa, solución y estado.	4
Sistemas operativos	Uso de PowerShell/CMD en Windows y comandos de terminal.	3
Servicios cloud (AWS)	Configuración de AWS con usuario IAM propio y creación de un recurso real.	3
Organización y control	Revisión del plan antes de aplicar y verificación del resultado.	3

Alcance del laboratorio

Qué es y qué no es este laboratorio
Es un ejercicio práctico individual en una cuenta de pruebas; no representa experiencia en producción. Un segundo ejercicio (instancia EC2 con módulo VPC) fue preparado como código, pero no se desplegó, por lo que no se incluye como resultado.

2. Levantamiento de información del laboratorio

Antes de ejecutar, se definieron los objetivos y los requisitos, igual que se haría al recibir un proyecto nuevo.

Objetivos

- Configurar AWS CLI con credenciales de un usuario IAM propio (no el usuario raíz).
- Entender el ciclo de trabajo de Terraform: init, fmt, validate, plan, apply, destroy.
- Crear y verificar un recurso real en AWS (bucket S3).
- Identificar errores frecuentes y aplicar buenas prácticas de seguridad.

Requisitos previos

Requisito	Detalle
Cuenta de AWS	Con acceso a la consola de IAM para crear un usuario y una clave de acceso.
AWS CLI v2	Verificado con aws --version (versión 2.36.24).
Terraform	Binario terraform.exe en la carpeta del proyecto o instalado en el PATH.
Editor de código	Visual Studio Code con la extensión HashiCorp Terraform (recomendado).
Terminal	PowerShell o CMD en Windows; cualquier shell en Linux/macOS.

Control de costos
El bucket S3 vacío cuesta prácticamente nada, pero recursos como EC2, NAT Gateway o VPN sí generan cargos. Al terminar cada práctica se debe ejecutar terraform destroy.

3. Ejecución del despliegue

3.1 Usuario IAM dedicado

Nunca se usa el usuario raíz para automatizar. Se creó el usuario terraform-lab, solo con acceso programático (sin consola), y sus permisos se asignaron mediante un grupo.

- En IAM > Grupos de usuarios se creó el grupo terraform-lab-admins y se le adjuntó la política AdministratorAccess.
- En IAM > Usuarios se creó terraform-lab, sin acceso a la consola, y se agregó al grupo.
- En Credenciales de seguridad se creó una clave de acceso (caso de uso: CLI) y se descargó el archivo .csv. El secreto solo se muestra una vez.

> **Permisos amplios solo para el laboratorio**
> AdministratorAccess concede control total de la cuenta. Es aceptable en un laboratorio aislado, pero en un entorno real debe reemplazarse por una política de mínimo privilegio.

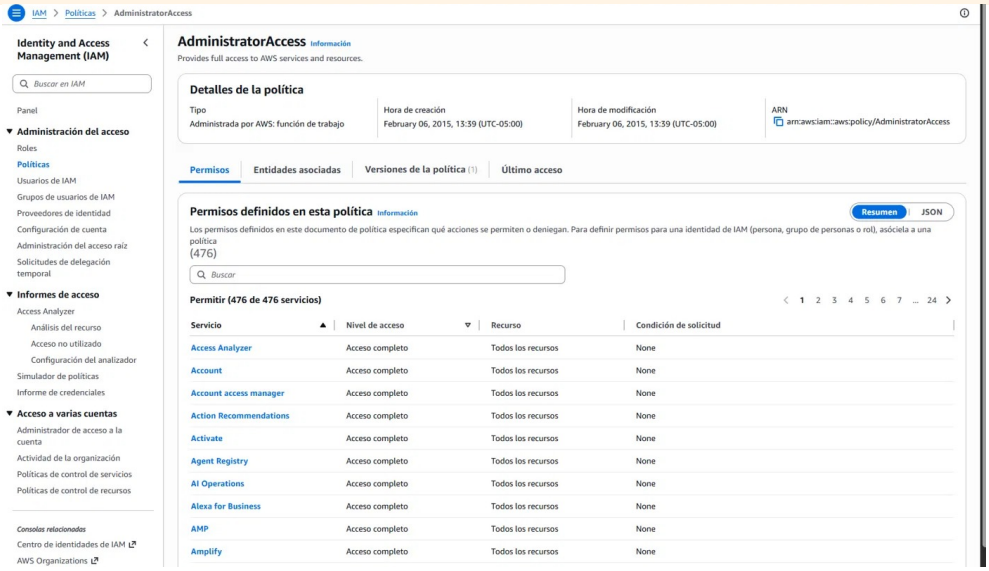


Figura 1. Política administrada AdministratorAccess (acceso completo a todos los servicios).

3.2 Configuración de AWS CLI

```
aws --version
aws configure
aws sts get-caller-identity
```

Con aws configure se ingresó el Access Key ID, el Secret Access Key, la región us-east-1 y el formato json. El comando sts get-caller-identity confirma la identidad: el Arn debe terminar en user/terraform-lab.

3.3 Proyecto Terraform

Archivos: main.tf (recursos), providers.tf (proveedor y versiones), variables.tf, terraform.tfvars (no subir a Git) y .terraform.lock.hcl (se genera con init; sí subir a Git).

```
# providers.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
provider "aws" { region = "us-east-1" }

# main.tf
resource "aws_s3_bucket" "example" {
  bucket = "my-tf-test-nicktf18102026"
  tags = {
    Name        = "My bucket"
    Environment = "Dev"
  }
}
```

El nombre del bucket es global
Los nombres de bucket S3 son únicos en todo AWS y solo admiten minúsculas, números y guiones. Un sufijo propio evita conflictos.

3.4 Flujo de trabajo de Terraform

Comando	Qué hace	Resultado en el laboratorio
terraform init	Descarga el provider y crea .terraform.lock.hcl	OK, hashicorp/aws v6.69.1
terraform fmt	Da formato estándar al código	OK, sin cambios
terraform validate	Revisa la sintaxis	OK, configuración válida
terraform plan	Muestra qué se creará, sin aplicar	OK, 1 to add, 0 to change, 0 to destroy
terraform apply	Crea los recursos (pide confirmar con yes)	1.er intento: InvalidBucketName. 2.º intento: OK, 1 added
terraform destroy	Elimina los recursos creados	Pendiente (ver sección 6)

Variante recomendada: guardar el plan, para aplicar exactamente lo que se revisó.

```
terraform plan -out=plan.tfplan
terraform apply plan.tfplan
```

3.5 Verificación

Tras el Apply complete!, el bucket apareció en Amazon S3 > Buckets de uso general, en us-east-1, y también se puede comprobar por CLI:

```
aws s3 ls
```

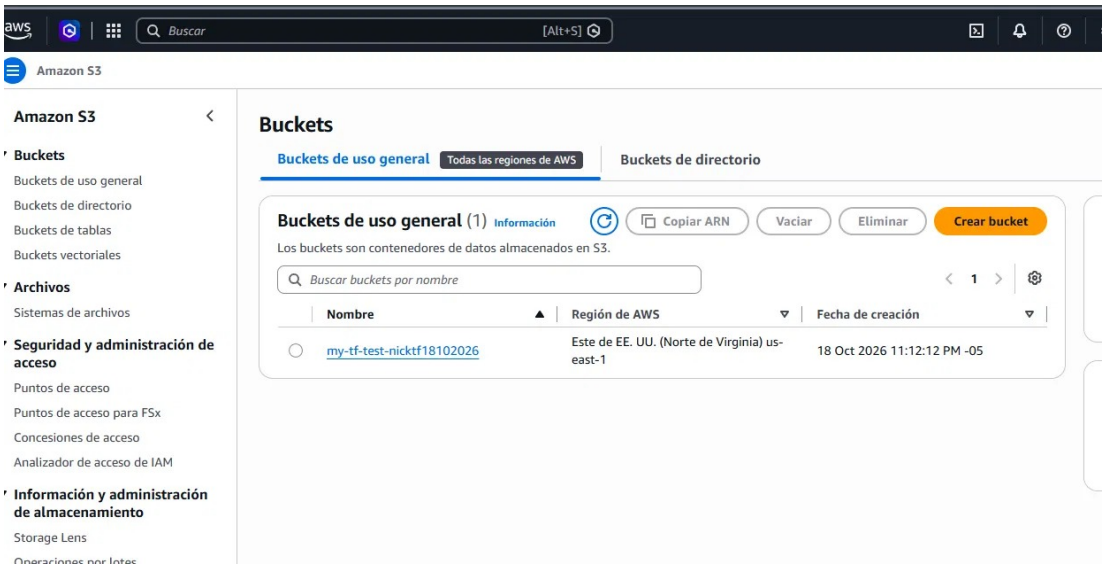


Figura 2. Bucket my-tf-test-nicktf18102026 creado por Terraform en us-east-1.

4. Registro de incidencias

Los errores encontrados se documentaron con síntoma, causa, solución y estado. Ambos quedaron resueltos.

ID	Síntoma	Causa	Solución	Estado
INC-01	SignatureDoesNotMatch al ejecutar aws sts get-caller-identity	El Secret Access Key no correspondía al Access Key ID (mal copiado o de otra clave).	Crear una clave nueva, repetir aws configure con ese par y desactivar la clave anterior. Copiar y pegar desde el .csv, sin espacios.	Resuelto
INC-02	InvalidBucketName en el primer terraform apply	El nombre del bucket tenía mayúsculas; S3 solo acepta minúsculas, números y guiones.	Cambiar el nombre a minúsculas y ejecutar terraform apply de nuevo.	Resuelto

Otros problemas típicos (guía de consulta)

Síntoma	Causa probable	Qué hacer
BucketAlreadyExists	El nombre ya lo usa otra cuenta en AWS	Elegir un nombre más único
No valid credential sources	No hay credenciales configuradas	Ejecutar aws configure y revisar sts get-caller-identity
Error acquiring the state lock	Otra ejecución tiene bloqueado el estado	Esperar o usar terraform force-unlock con cuidado
Provider no instalado	Falta inicializar	Ejecutar terraform init

5. Buenas prácticas

Control de versiones

- Guardar .terraform.lock.hcl en Git para fijar la versión del provider.
- Crear un .gitignore que excluya .terraform/, *.tfstate, *.tfstate.* y *.tfvars.

Seguridad en AWS

- No compartir el Secret Access Key. Si se expone, desactivarlo en IAM y crear otro.
- Eliminar las claves de acceso que no se usan.
- Activar MFA en la cuenta y en los usuarios con acceso a la consola.
- Configurar un presupuesto en AWS Budgets con alertas por correo.
- Reemplazar AdministratorAccess por una política de mínimo privilegio cuando el laboratorio funcione.
- Rotar las claves periódicamente o migrar a credenciales temporales (IAM Identity Center).

Terraform

- Ejecutar siempre plan antes de apply y leerlo.
- Usar variables en lugar de valores fijos y documentarlas en variables.tf.
- Etiquetar los recursos (Environment, Owner, Project) para controlar costos.
- Fijar versiones del provider y de los módulos.

6. Cierre y siguientes pasos

Limpieza: ejecutar terraform destroy, escribir yes y verificar en la consola de S3 que el bucket ya no existe. Este paso figura como pendiente en el laboratorio.

Siguientes pasos de aprendizaje

- Mover el estado a un backend remoto en S3 con bloqueo (requiere Terraform 1.10 o superior; el bucket de estado debe existir antes y tener versionado y cifrado).
- Desplegar el ejercicio preparado de EC2 con módulo VPC, fijando la versión del módulo y validando el filtro de AMI.
- Reemplazar AdministratorAccess por una política de mínimo privilegio.

Ejercicio preparado: EC2 + VPC (no desplegado)

Como siguiente práctica se preparó un segundo main.tf con el módulo oficial terraform-aws-modules/vpc/aws, una consulta dinámica de AMI (data source) y una instancia EC2 con tipo definido en variables.tf. Está como código; aún no se ha aplicado.

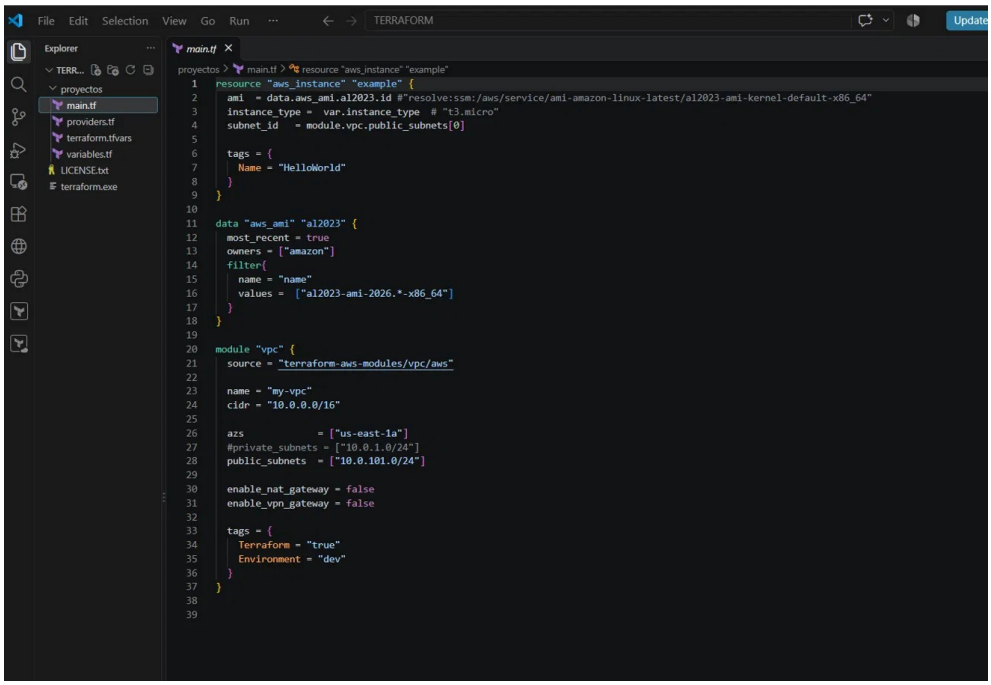


Figura 3. main.tf ampliado: instancia EC2, data source de AMI y módulo VPC.

> **Mejoras sugeridas para ese código**
> Fijar la versión del módulo (version = "~> 6.0" o la vigente), revisar que el patrón del filtro de AMI coincida con imágenes existentes (si no devuelve resultados, el plan falla) y ejecutar terraform init otra vez al añadir un módulo nuevo.

Referencia rápida

Comando	Para qué sirve
aws sts get-caller-identity	Ver qué identidad está usando la CLI
aws s3 ls	Listar buckets
terraform init / fmt / validate	Inicializar, formatear y validar
terraform plan / apply	Previsualizar y aplicar cambios
terraform state list	Listar recursos gestionados
terraform destroy	Eliminar todo lo creado